

CONVOLUTIONAL NEURAL NETWORKS

- **CNN** stands for **Convolutional Neural Network**, which is a type of **deep learning model** commonly used in the **field of computer vision**. CNNs are particularly effective in tasks related to **image recognition and classification**.
- Convolution neural network (**also known as *ConvNet* or *CNN***) is a type of **feed-forward neural network** used in tasks like **image analysis**.
- This CNN Algorithm **take the image as an input** and **learn the various features of the image** through **filters**.

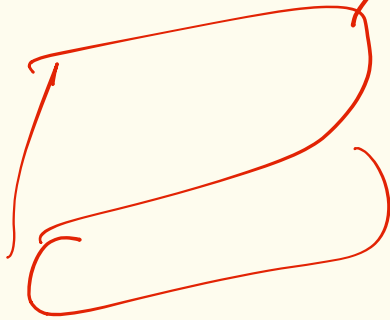


- It consists of 3 Layers. Those are
 - 1. Convolution Layer
 - 2. Pooling Layer
 - 3. Fully Connected Layer or Dense Layer
- Convolutional neural network (CNN) is a deep learning neural network designed for processing structured arrays of data such as images.
- A CNN is a feed-forward neural network, often with up to 20 or 30 layers.
- CNN represents the input data in the form of multidimensional arrays.

HN $\rightarrow$ 1D

Range $\rightarrow$ 3M

2D



1D

3M

1D

$\rightarrow 0$

$\rightarrow 0$

$\rightarrow 0$

$\rightarrow 0$

KNN

HN
1D

- It works well for a large number of labeled data. CNN extract the each and every portion of input image, which is known as receptive field. It assigns weights for each neuron based on the significant role of the receptive field.
- As soon as we see an image, our brain starts categorizing it based on the colour, shape, etc.. Similar thing can be done through machines even after a rigorous training.
- But the difficulty is there is a huge difference in what humans interpret and what machine does.



- For a machine, the image is merely an array of pixels. There is a unique pattern included in each object present in the image and the computer tries to find out these patterns to get the information about the image.
- Machines can be trained giving tons of images to increase its ability to recognize the objects included in a given input image.
- Most of the digital companies have opted for CNNs for image recognition, some of these include Google, Amazon, Instagram, Interest, Facebook, etc.



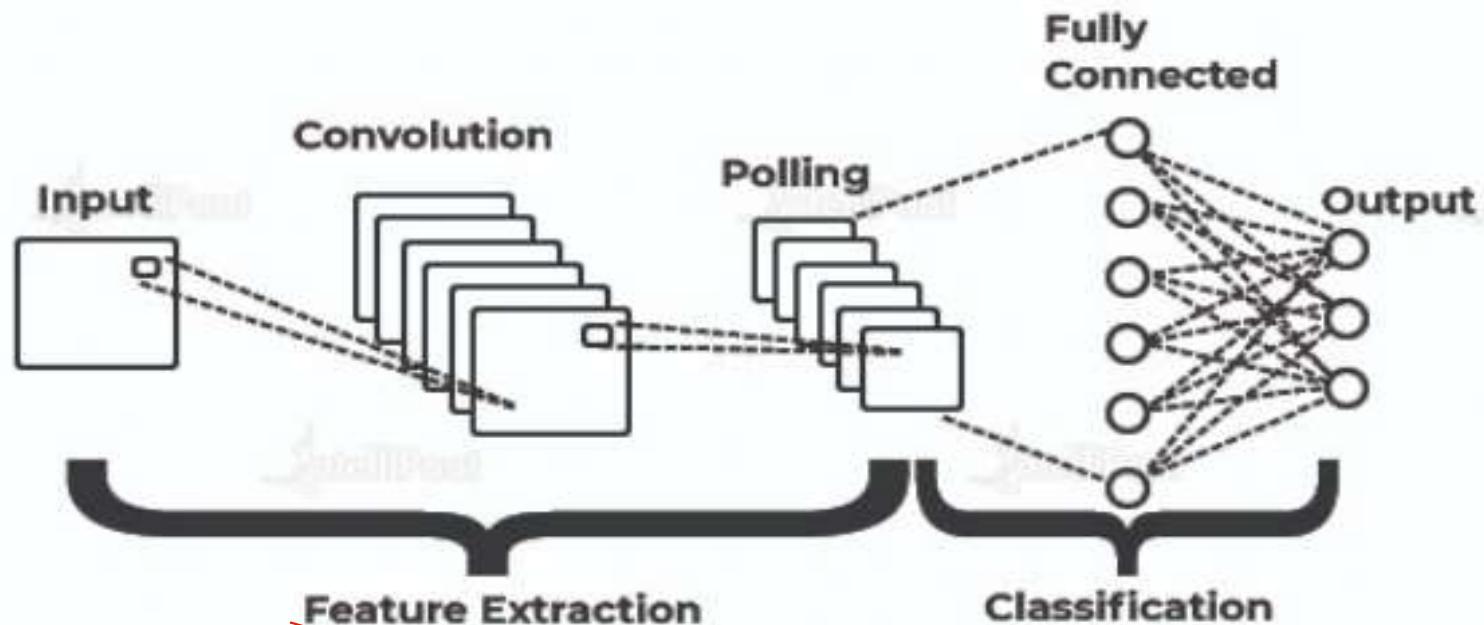
- Hence, we define a convolutional neural network as: "A neural network consisting of multiple convolutional layers which are used mainly for image processing, classification, segmentation and other correlated data".
- CNNs are designed to solve problems with real time applications of image recognition and analysis, including image classification, drug discovery, and health risk assessments. CNNs also help provide depth estimation for self-driving cars.



ARCHITECTURE OF CNN

➤ CNN consists of 3 Layers. Those are

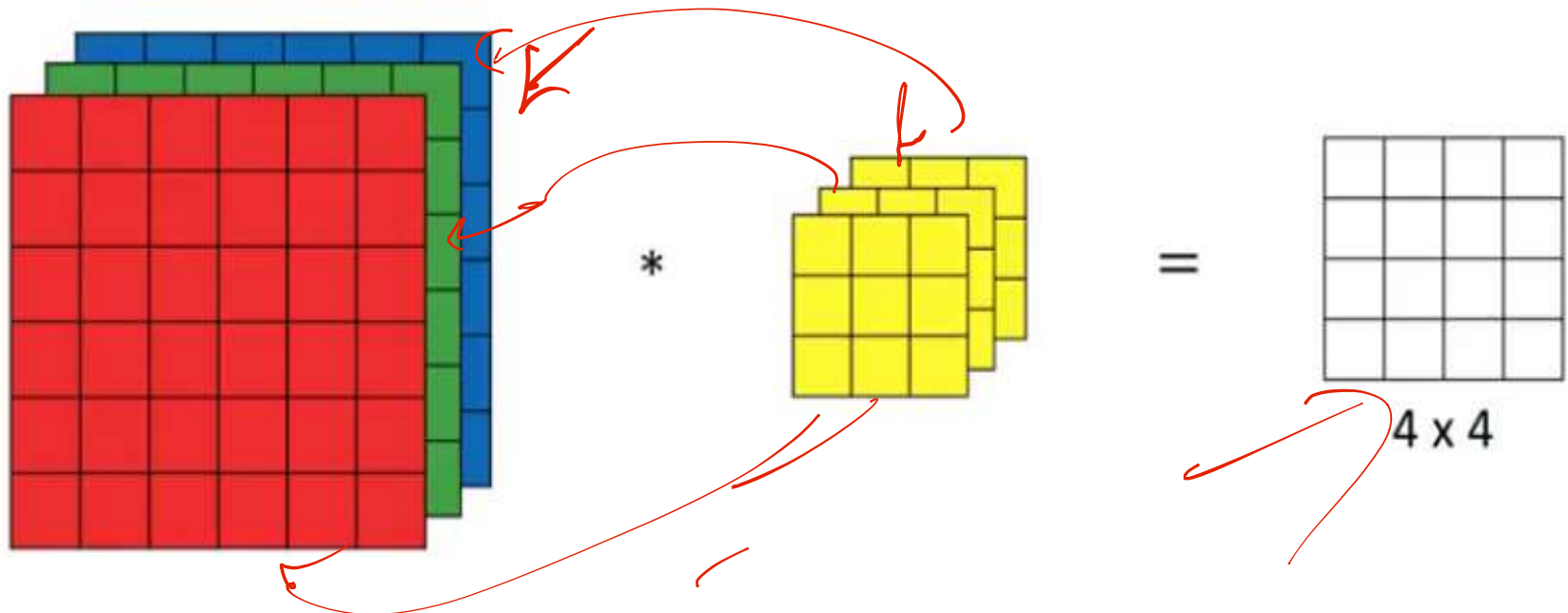
- 1. Convolution Layer
- 2. Pooling Layer
- 3. Fully Connected Layer or Dense Layer



CNN Architecture

- CNN takes an **image as input**, which is classified and process under a certain category such as **dog, cat, lion, tiger, etc.**
- The computer sees an image as an **array of pixels** and depends on the resolution of the image.
- Based on image resolution, it will see as **$h * w * d$** ,
 - where **h** = height
 - **w** = width and
 - **d** = dimension. / 
- For example, An **RGB image** is **$6 * 6 * 3$** array of the matrix, and the **gray scale image** is **$4 * 4 * 1$** array of the matrix.

- In CNN, **each input image** will pass through a sequence of convolution layers along **with pooling**, fully connected layers, filters (Also known as kernels). *T_{Reduce}*
- After that, we will apply the Activation function to **classify an object with probabilistic values**.



LAYERS IN CNN

1. CONVOLUTION OPERATION

- 1. The Convolution Layer: Convolution operation focuses on extracting/preserving important features from the input.
- It means the network will learn specific patterns within the picture and will be able to recognize it everywhere in the picture.
- Convolution operation allows the network to detect horizontal and vertical edges of an image and then based on those edges build high-level features in the following layers of neural network.



➤ Convolution operation uses three parameters:

- Input image, *↑*
- Feature detector and *Filter/Kernel*
- Feature map. *→ o/p*

➤ It uses the mathematical operation which takes two inputs such as image matrix and a kernel or filter.

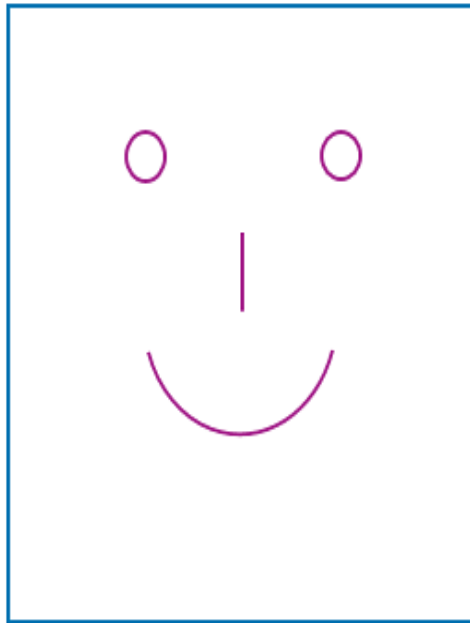
➤ These filters slide over the input data, performing element-wise multiplications and summations to produce feature maps. Each filter specializes in detecting a specific pattern or feature within the input.

SOP

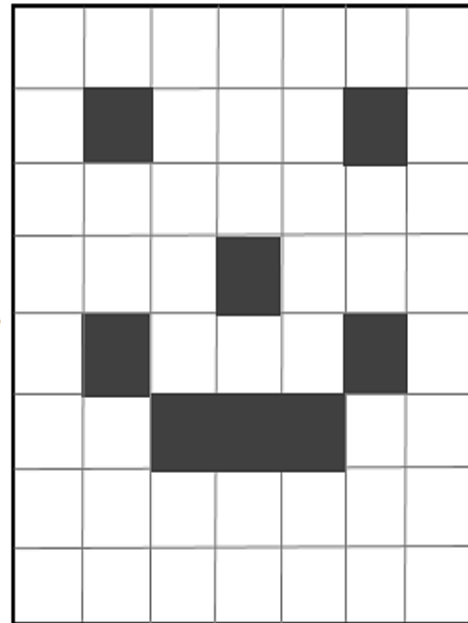


- Input matrix can be pixel values of a gray scale image whereas a filter is a relatively small matrix that detects edges by darkening areas of input image .
- There can be different types of filters depending upon what type of features we want to detect.
- Eg: Vertical, Horizontal, or Diagonal, etc.
- Input image is converted into binary 1 and 0. In this Image, how CNN recognizes an image: The convolution operation shown in the figure is known as the feature detector of a CNN.





Real Image



Represented in the form of
black and white pixels

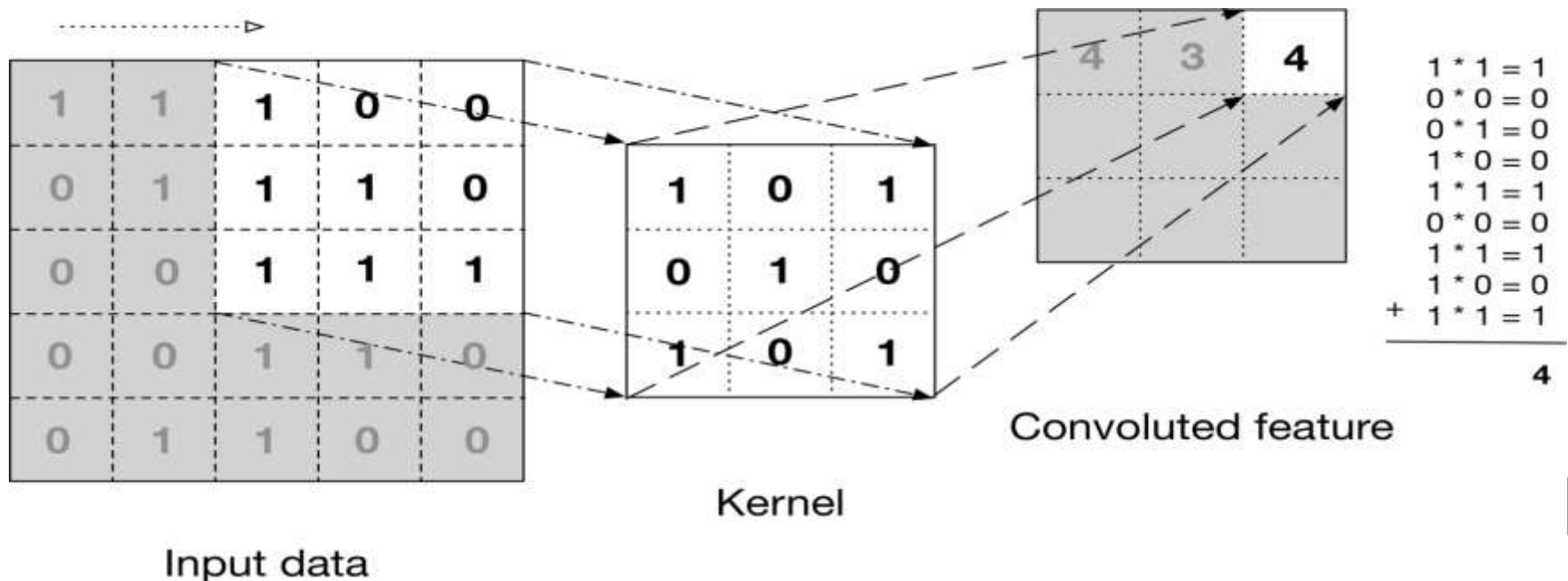


0	0	0	0	0	0	0
0	1	0	0	0	1	0
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	1	0	0	0	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0

Image represented in the
form of a matrix of numbers



- A feature detector, also known as **a kernel or a filter**, will **traverse over the image's receptive fields**, checking for the **presence of the feature**.
- For simplicity, let's stick with **grayscale images** as we try to understand how CNNs work.



0	1	0	1	0	1	0
1	1	1	1	1	1	1
0	1	1	1	1	1	0
1	1	1	1	1	1	1
0	1	1	1	1	1	0

1	0	1
0	1	0
1	0	1

3x3

$$n - f + 1$$

$$7 - 3 + 1 \Rightarrow 5 \times 5$$

$$\frac{7 - 3 + 1}{2}$$

$$\frac{2 \times 4}{2} + 1 \Rightarrow \underline{3 \times 3}$$

- The input to a convolution can be raw data or a feature map output from another convolution.
- The **given image shows what a convolution is**. We take a **filter/kernel(3×3 matrix)** and **apply it to the input image** to get the **convolved feature**. This convolved feature is passed on to the next layer.

1	1	1	0	0
0	1	1	1	0
0	0	1 _{x1}	1 _{x0}	1 _{x1}
0	0	1 _{x0}	1 _{x1}	0 _{x0}
0	1	1 _{x1}	0 _{x0}	0 _{x1}

Image

4	3	4
2	4	3
2	3	4

Convolved
Feature



- Convoluting a **5x5x1 image** with a **3x3x1 kernel** to get a **3x3x1 convolved feature**
- Generally, an image can be considered as **a matrix** whose elements are numbers **between 0 and 255**. The size of image matrix is:
$$\text{image height} * \text{image width} * \text{number of image channels}.$$
- A **gray scale image has 1 channel**, where a **colour image has 3 channels**.
- Let's pretend the input is a **color picture**, which is made up of a **3D matrix of pixels**.



- This implies the **input will have three dimensions**:
 - **height, width, and depth**, which match the **RGB color space of a picture**. Here we try to decompose RGB to a multidimensional layer and apply a filter to each layer.
- The dimension of the image matrix is **$h \times w \times d$** .
- The dimension of the filter is **$f_h \times f_w \times d$** .

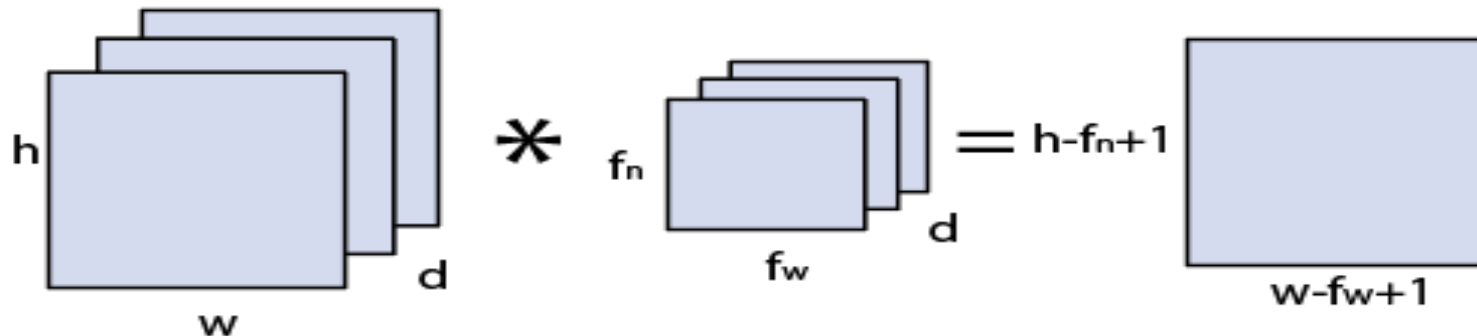


Image matrix multiplies kernel or filter matrix



Output Size

$$\left\{ \begin{array}{l} \text{Image} \cdot m \times n \\ \text{Kernel} \rightarrow f \times f \end{array} \right\} \Rightarrow m - f + 1 \quad \text{Stride} \Rightarrow 1$$

$$\left\{ \begin{array}{l} 39 \times 39 \times 3 \\ f \Rightarrow 3 \times 3 \\ \# \text{ filter} = 10 \end{array} \right\} \Rightarrow \begin{array}{l} 37 \times 37 \times 10 \\ \downarrow \\ f \Rightarrow 5 \times 5 \\ \# \text{ filter} \Rightarrow 20 \\ \text{o/p size} \Rightarrow \\ 33 \times 33 \times 20 \end{array}$$

$$\underline{\underline{\text{Stride} = 2}}$$

$$\left(\frac{n-f}{s} + 1 \right) \times \left(\frac{h-f}{s} + 1 \right) \times \text{depth}$$

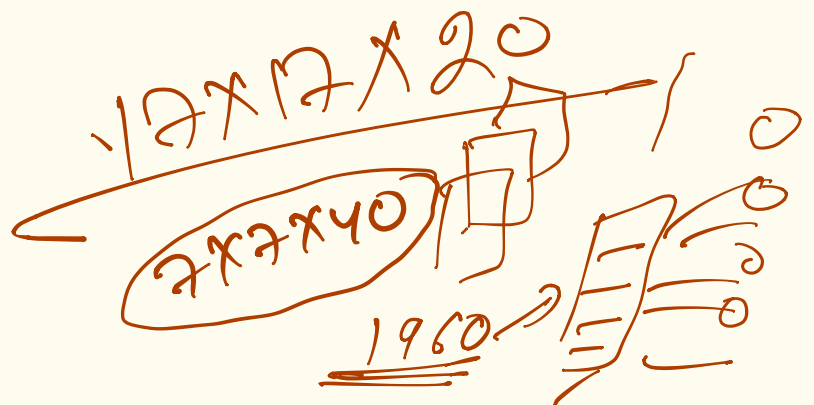
$\uparrow$
 $\#$
 filters

$$37 \times 37 \times 10$$

$$\# 20$$

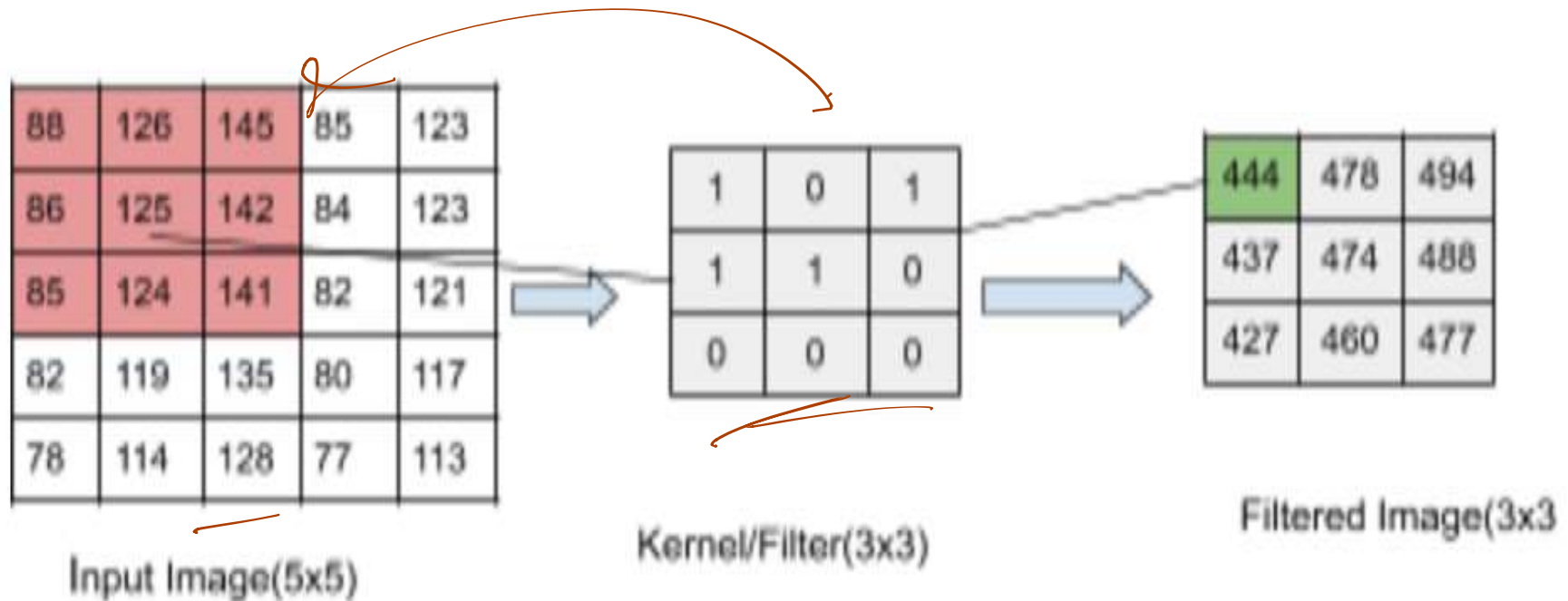
$$\text{filter} \Rightarrow 5 \times 5$$

$$\text{Stride} = 2$$



- Here, Convolution is an **element-wise multiplication**. First, The computer will **scan a part of the image**, usually with a dimension of **3x3** and **multiplies it to a filter**.
- The **output** of the element-wise multiplication is called a **feature map**. This **step is repeated** until **all the image is scanned**.
- Note that, after the convolution, the **size of the image is reduced**.





$$\begin{aligned}
 & (88*1 + 126*0 + 145*1) + (86*1 + 125*1 + 142*0) + \\
 & (85*0 + 124*0 + 141*0) \\
 & = (88 + 145) + (86 + 125) \\
 & = 233 + 211 \\
 & = 444
 \end{aligned}$$

Local Receptive Field

Filtered pixel value

Input Slice (6 x 6)

Receptive
Field

20	24	11	12	16	19
19	17	20	23	15	9
21	40	25	13	14	8
9	18	8	6	11	22
31	3	7	9	17	23
20	12	3	11	19	30

Sobel Kernel
(3 x 3)

1	0	-1
2	0	-2
1	0	-1

*

=

Output (4 x 4)

3			

Sample calculation for first filter location

$$(20 \times 1) + (24 \times 0) + (11 \times -1) + (19 \times 2) + (17 \times 0) + (20 \times -2) + (21 \times 1) + (40 \times 0) + (25 \times -1) = 3$$

20	24	11	12	16	19
19	17	20	23	15	9
21	40	25	13	14	8
9	18	8	6	11	22
31	3	7	9	17	23
20	12	3	11	19	30

*

=

3	27		

Sample calculation for second filter location

$$(24 \times 1) + (11 \times 0) + (12 \times -1) + (17 \times 2) + (20 \times 0) + (23 \times -2) + (40 \times 1) + (25 \times 0) + (13 \times -1) = 27$$

- Different Types of Kernels or Filters: Convolution of a picture with totally different filters will perform operations like edge detection, blur and sharpen by applying filters.
- The below example shows numerous convolution images once applying different types of filters (kernels).

1	0	-1
1	0	-1
1	0	-1

Vertical

1	1	1
0	0	0
-1	-1	-1

Horizontal

1	0	-1
2	0	-2
1	0	-1

Sobel filter

3	0	-3
10	0	-10
3	0	-3

Scharr filter



<i>Original</i>	<i>Gaussian Blur</i>	<i>Sharpen</i>	<i>Edge Detection</i>
$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$
			

➤ Important Parameters in CNN are:

- 1. Stride: Stride determines how many pixels the kernel shifts over the input at a time.

$p=1 \Rightarrow 5 \times 5 / 3 \times 3$

$p=2$

$7 \times 7 \rightarrow 5 \times 5 \Rightarrow p=1$

$9 \times 9 \rightarrow 5 \times 5 \Rightarrow 3 \times 3$

5×5

5	4	3	2	1
5	5	3	2	1
5	4	5	2	1
5	4	3	5	1
5	4	5	5	1

Input Image(x)
(n×n)

1	1	1
0	0	0
1	1	1

Filter
(f×f)

=

26	20	14
25	22	15
28	25	19

Output Image (y) dimension
(n - f + 1) × (n - f + 1)

Output Image (y) dimension

$$\left(\frac{n-f}{s} + 1\right) \times \left(\frac{n-f}{s} + 1\right)$$

Strides (s) = 1

$$\frac{n+2p-f}{s} + 1$$

Strides = 2

5	4	3	2	1
5	5	3	2	1
5	4	5	2	1
5	4	3	5	1
5	4	5	5	1

Input Image(x)

(n x n)

5 x 5

*

1	1	1
0	0	0
1	1	1

Filter

(f x f)

3 x 3

=

26	14
28	19

Output Image (y1)

$(\frac{n-f}{s} + 1) \times (\frac{n-f}{s} + 1)$

$(\frac{5-3}{2} + 1) \times (\frac{5-3}{2} + 1)$

2 x 2



Strides = 3

<table border="1"><tr><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td></tr><tr><td>5</td><td>5</td><td>3</td><td>2</td><td>1</td></tr><tr><td>5</td><td>4</td><td>5</td><td>2</td><td>1</td></tr><tr><td>5</td><td>4</td><td>3</td><td>5</td><td>1</td></tr><tr><td>5</td><td>4</td><td>5</td><td>5</td><td>1</td></tr></table>	5	4	3	2	1	5	5	3	2	1	5	4	5	2	1	5	4	3	5	1	5	4	5	5	1	*	<table border="1"><tr><td>1</td><td>1</td><td>1</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	1	1	1	0	0	0	1	1	1	=	<table border="1"><tr><td>26</td></tr></table>	26
5	4	3	2	1																																			
5	5	3	2	1																																			
5	4	5	2	1																																			
5	4	3	5	1																																			
5	4	5	5	1																																			
1	1	1																																					
0	0	0																																					
1	1	1																																					
26																																							
Input Image(x) (n x n) 5 x 5		Filter (f x f) 3 x 3		Output Image (y1) $(\frac{n-f}{s} + 1) \times (\frac{n-f}{s} + 1)$ 1 x 1																																			

When the convolution window continues to slide three columns to the right on the input, there is no output because the input element cannot fill the window (unless we add another column of padding).

2. Padding: Padding is a technique used to preserve the spatial dimensions of the input image after convolution operations on a feature map. Padding involves adding extra pixels around the border of the input feature map before convolution.

$$\frac{m}{2}$$

$$p =$$



Strides =3

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	5	4	3	2	1	0	0
0	0	5	5	3	2	1	0	0
0	0	5	4	5	2	1	0	0
0	0	5	4	3	5	1	0	0
0	0	5	4	5	5	1	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

Input Image(x)
(n x n) with p=2
5 x 5 with p=2 (9 x 9)

*

1	1	1
0	0	0
1	1	1

Filter
(f x f)
3 x 3

=

5	9	1
10	22	2
5	14	1

Output Image (y1)
 $\left(\frac{n+2p-f}{s} + 1\right) \times \left(\frac{n+2p-f}{s} + 1\right)$
3 x 3

Dimension of the feature map as a function of the input image size(W), feature detector size(F), Stride(S) and Zero Padding on image(P) is

$$(W - F + 2P) / S + 1$$

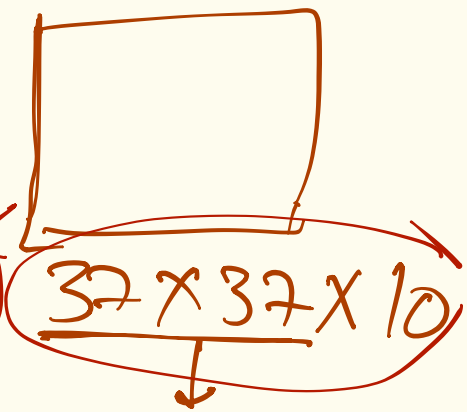
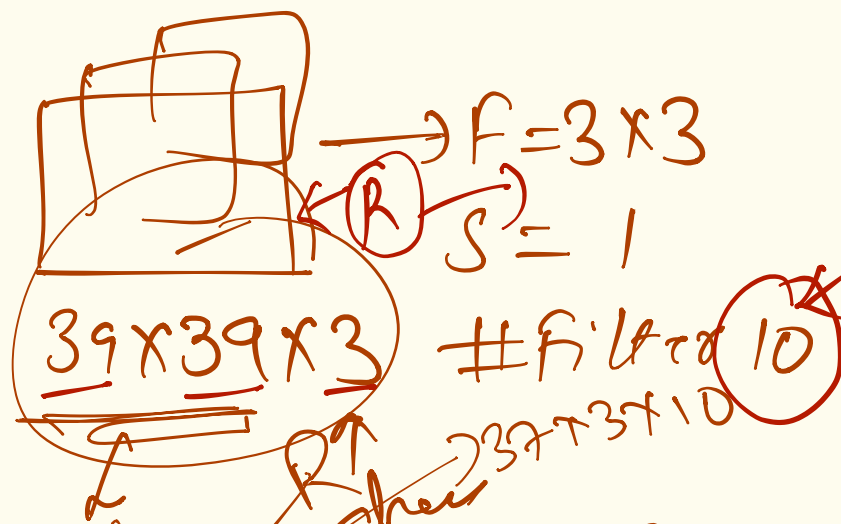
Input image size W in our case is 5.

Feature detector or receptive field size is F, which in our case is 3

Stride (S) is 1, and the amount of zero padding used (P) on the image is 0.

so, our feature map dimension will $(5 - 3 + 0) / 1 + 1 = 3$.

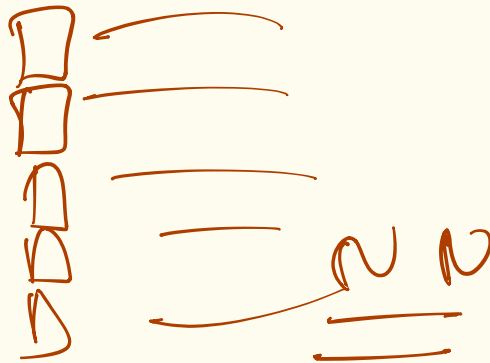
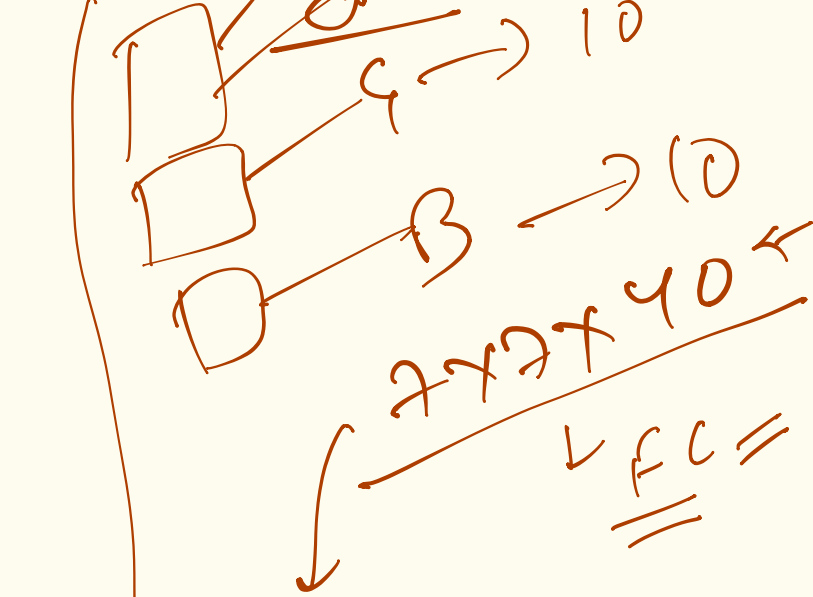
so feature map will a 3*3 matrix with three channels(RGB).



$P = 0$
 $\# \text{ filter} = 20$

$17 \times 17 \times 20$

$f = 5 \times 5$
 $S = 2$
 $\# \text{ fill } 40$



ACTIVATION FUNCTION USED IN CNN

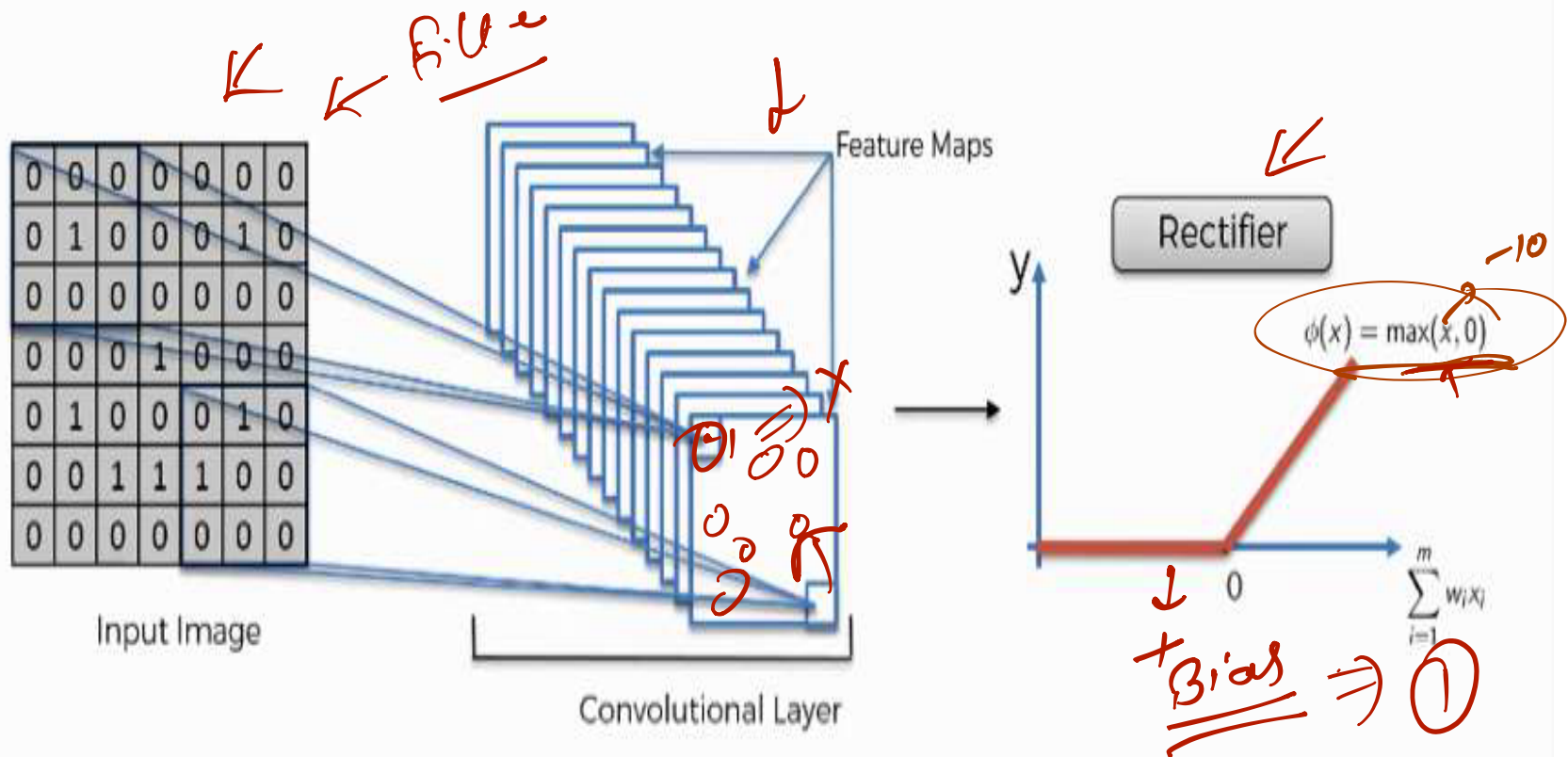
- The **output volume of the Convolution layer** is fed to **an element wise activation function**, commonly a **Rectified-Linear Unit (ReLU)**.
- After **each CONV layer in a CNN**, we **apply a nonlinear activation function**, such **as ReLU**. The **Rectified Linear Unit, or ReLU**, is not a separate component of the convolutional neural networks' process.
- The **ReLU layer will determine whether an input node will 'fire' given the input data**.



ACTIVATION FUNCTION USED IN CNN

- This 'firing' signals whether the convolution layer's filters have detected a visual feature.
- The dimensions of the volume are left unchanged.
- The ReLU (short for rectified linear units) layer commonly follows the convolution layer. i.e. the ReLU layer allows the convnet to account for situations in which the relationship between the pixel value inputs and the convnet output is not linear.





The purpose of applying **the rectifier function** is to **increase the non-linearity in our images**. The **ReLU function** takes a **value x** and **returns 0** if x is negative and **x** if x is positive.

Input

<u>-249</u>	-91	-37
250	-134	101
27	61	-153

ReLU

<u>0</u>	0	0
250	0	101
27	61	0

①

As you can see from the graph, the ReLU function is nonlinear. In this layer, the **ReLU function is applied to each point in the feature map**. The result is a feature map **without negative values**.

POOLING LAYER ↩

- **Pooling layers** are one of the building blocks of **Convolutional Neural Networks**.
- Where **Convolutional layers** **extract features from images**, **Pooling layers** **consolidate the features** learned by CNNs.
- we use a **pooling function** to modify the output of the layer further.
Its purpose is to gradually shrink the representation's spatial dimension to minimize the number of parameters and computations in the network.
- **Pooling layers** are used to reduce the dimensions of the feature maps.

- i.e, The pooling layer summarizes the features **present in a region of the feature map** generated by a **convolution layer**. This makes the model **more robust to variations** in the position of the features in the input image.
- The size of the pooling operation or filter is **smaller than the size of the feature map**. This means that the pooling layer will **always reduce the size of each feature map by a factor of 2**.
- **Eg: Each Dimension is halved**, reducing the number of pixels or values in **each feature map** to **one quarter the size**.



- For example: a pooling layer applied to a feature map of 6×6 (36 pixels) will result in an output pooled feature map of 3×3 (9 pixels).

The **pooling operation**, also called **subsampling** is used to reduce the dimensionality of feature maps from the **convolution operation**.

- Pooling layers, also known as **down sampling**, conducts dimensionality reduction, reducing the number of parameters in the input.



max

2x2

Window
of 3x3
Stride
↓
2

10	20	30	40	50	60
1	5	11	12	13	14
2	7	32	33	34	15
3	8	24	25	26	16
4	9	30	23	31	17
5	10	19	20	22	18

20	40	60
1	5	11
2	7	32

Pooling
3x3

- ① Maxpooling
- ② Avg Pooling
- ③ global pooling layer

20	40	60
1	5	11
2	7	32

6x6

2x2
Stride = 2

Avg Pool 2

Global Pooling

- Similar to the **convolutional layer**, the **pooling operation** sweeps a **filter across the entire input**, but the difference is that this **filter does not have any weights**. Instead, the **kernel** applies an aggregation function to the values within the receptive field, populating the output array.

- **Types of Pooling Layer:**

- 1. Max Pooling ←
- 2. Average Pooling
- 3. Min Pooling } →

By default size
of pooling
filter 2×2
with
stride = 2



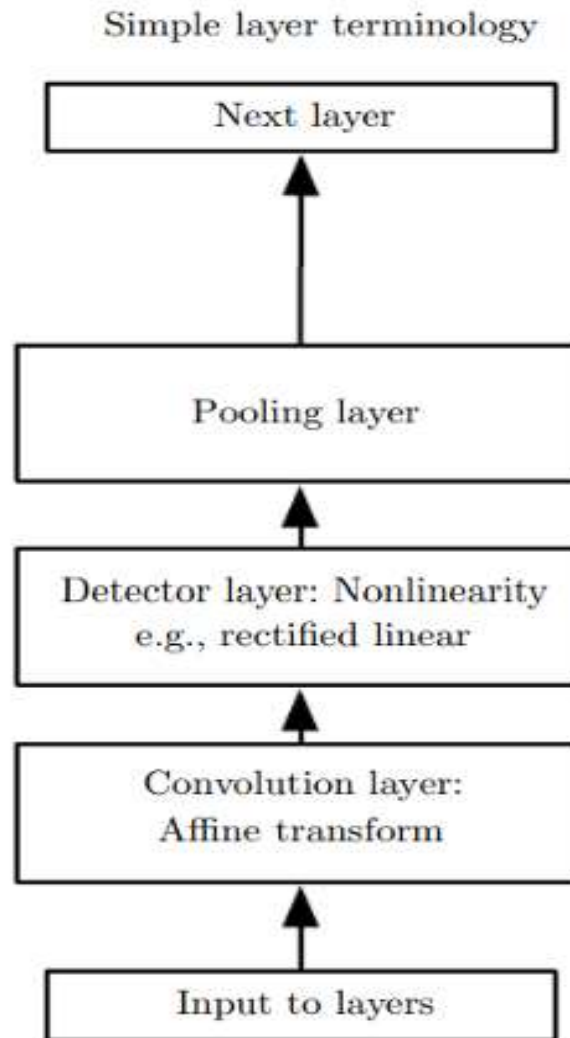


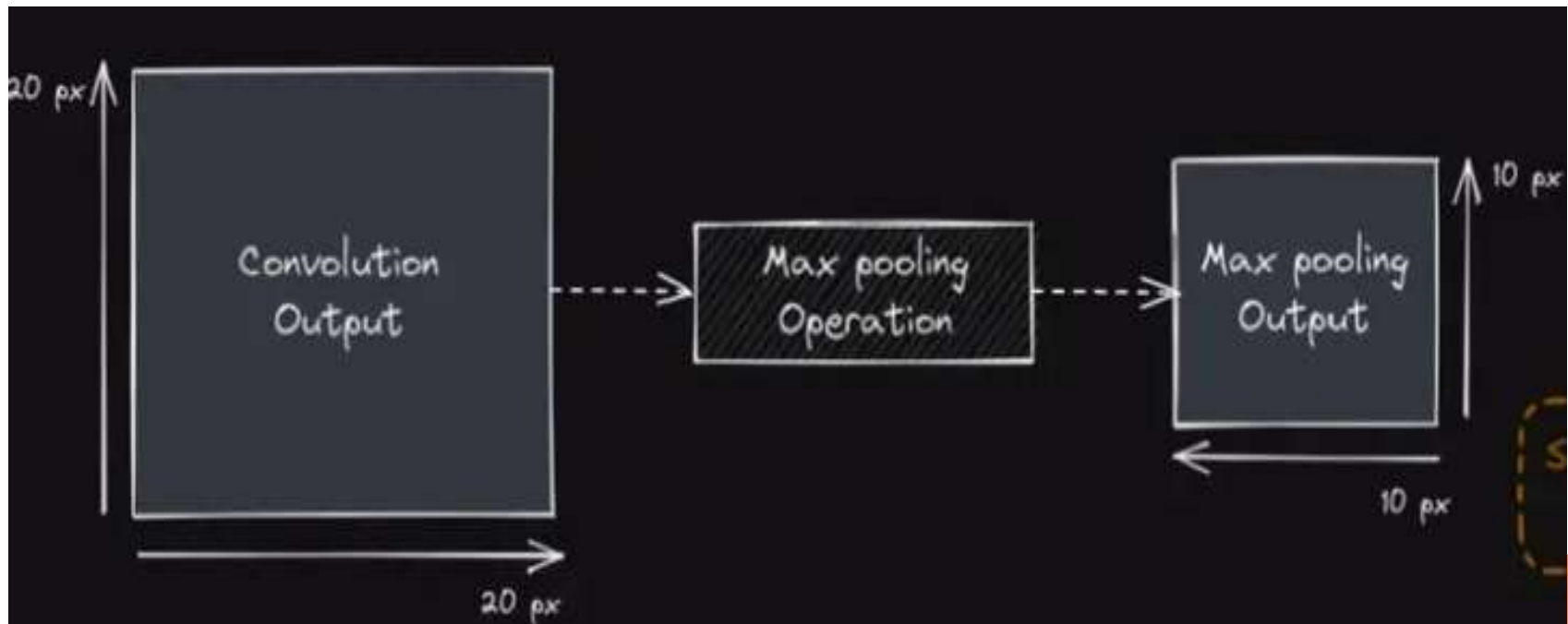
Fig: Components of a typical Convolutional Neural Network

1. Max Pooling: Finally, we can say it selects the maximum valued element from the region captured by the filter in any feature map.

This works by selecting the maximum value from every pool. Max Pooling retains the most prominent features of the feature map, and the returned image is sharper than the original image.

- i.e, if we talk about image processing the max-pooling helps to extract the sharpest features on the image.
- The Max Pooling layer summarizes the features in a region represented by the maximum value in that region. Max Pooling is more suitable for images with a dark background as it will select brighter pixels in a region of the input image.

Kernel/filter is the size of a matrix that is applied over the complete input data. As the filter moves across the input, it selects the pixel with the maximum value to send to the output array.



For Eg:

stride 1

Input

0	1	2
3	4	5
6	7	8

2 x 2 MAX
Pooling

Output

4	5
7	8

$$4 > 3 > 1 > 0 = 4$$

Input

0	1	2
3	4	5
6	7	8

2 x 2 MAX
Pooling

Output

4	5
7	8

$$5 > 4 > 2 > 1 = 5$$

Input

0	1	2
3	4	5
6	7	8

2 x 2 MAX
Pooling

Output

4	5
7	8

$$7 > 6 > 4 > 3 = 7$$

Input

0	1	2
3	4	5
6	7	8

2 x 2 MAX
Pooling

Output

4	5
7	8

$$8 > 7 > 5 > 4 = 8$$

- Mathematical Formulae: The **pooling layer** requires 2 hyperparameters

1. Kernel/filter size F and

2. Stride S.

- On applying the **pooling layer** over the input volume, output dimensions of output volume will be

$$\left\{ ((n-f)/s + 1) \times ((n-f)/s + 1) \right.$$



Max Pooling Input & Filter

4 x 4 Input

4	3	8	5
9	1	3	6
6	3	5	3
2	5	5	2

2 x 2 Filter

Max Pooling Operation

4 x 4 Input

4	3	8	5
9	1	3	6
6	3	5	3
2	5	5	2

Pool 1
Output

9

Max Pooling Output

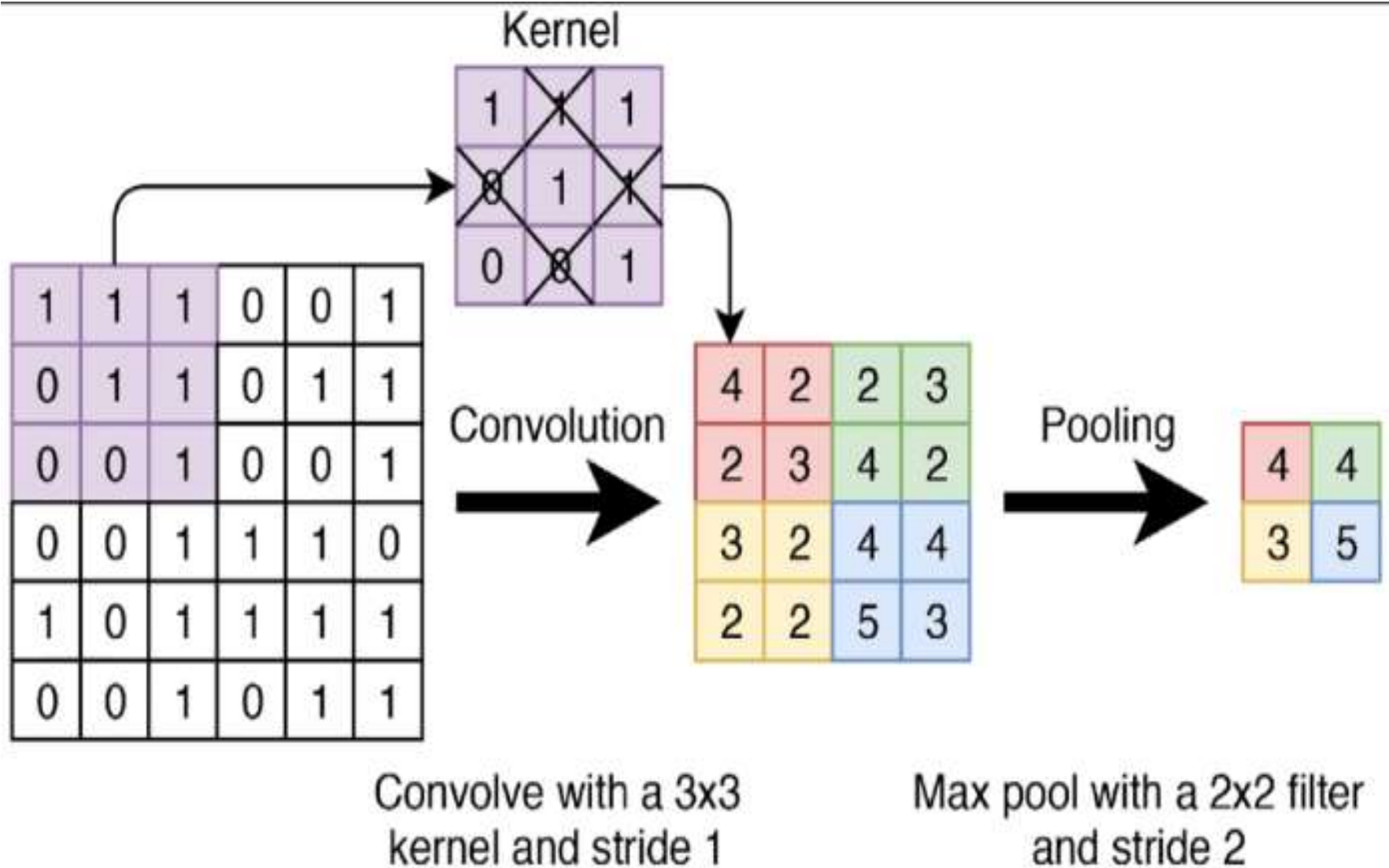
4 x 4 Input

4	3	8	5
9	1	3	6
6	3	5	3
2	5	5	2

Final Output

9	8
6	5

For $n \times n$ input and $f \times f$ filter with stride s , the output dimensions are: $\left(\frac{n-f}{s} + 1\right) \times \left(\frac{n-f}{s} + 1\right)$



- 2. Average pooling : As the filter moves across the input, it calculates the average value within the receptive field to send to the output array.
- This pooling layer works by getting the average of the pool. Average pooling retains the average values of features of the feature map.
- Average pooling method smooth's out the image and hence the sharp features may not be identified when this pooling method is used.



- With average pooling, the **harsh edges** of a picture are **smoothened**, and this type of pooling layer can be used **when harsh edges can be ignored**.

Rectified feature map

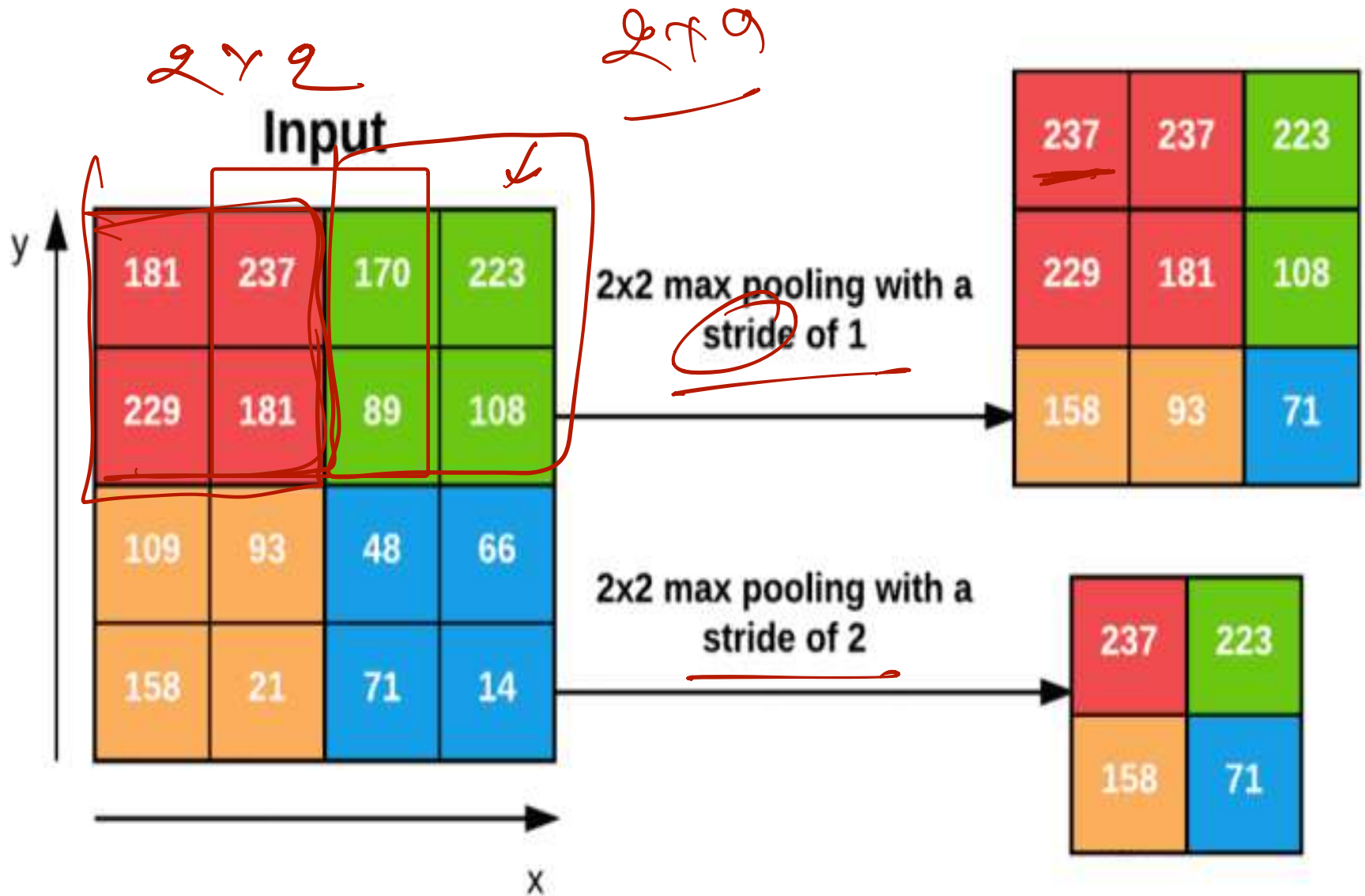
1	4	2	7
2	6	8	5
3	4	0	7
1	2	3	1

Average pooling with 2×2 filters and stride 2

Pooled feature map

3.2	6.2
2.5	2.7

$$\text{Average}(3, 4, 1, 2) = 2.5$$

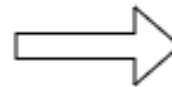


- 3. Min pooling : In this type of pooling, the summary of the features in a region is represented by the minimum value in that region.
- It is mostly used when the image has a light background .

255	167	25	46
2	70	251	58
8	59	68	90
4	125	201	126

Filter Size:

2x2



Stride: 2

2	25
4	68



Pool size → → Stride

-4	0	-2	4	1
3	1	0	2	1
1	0	1	1	1
4	6	5	1	0
-1	2	0	0	0

Features

Max Pooling

3	4
6	5

Min Pooling

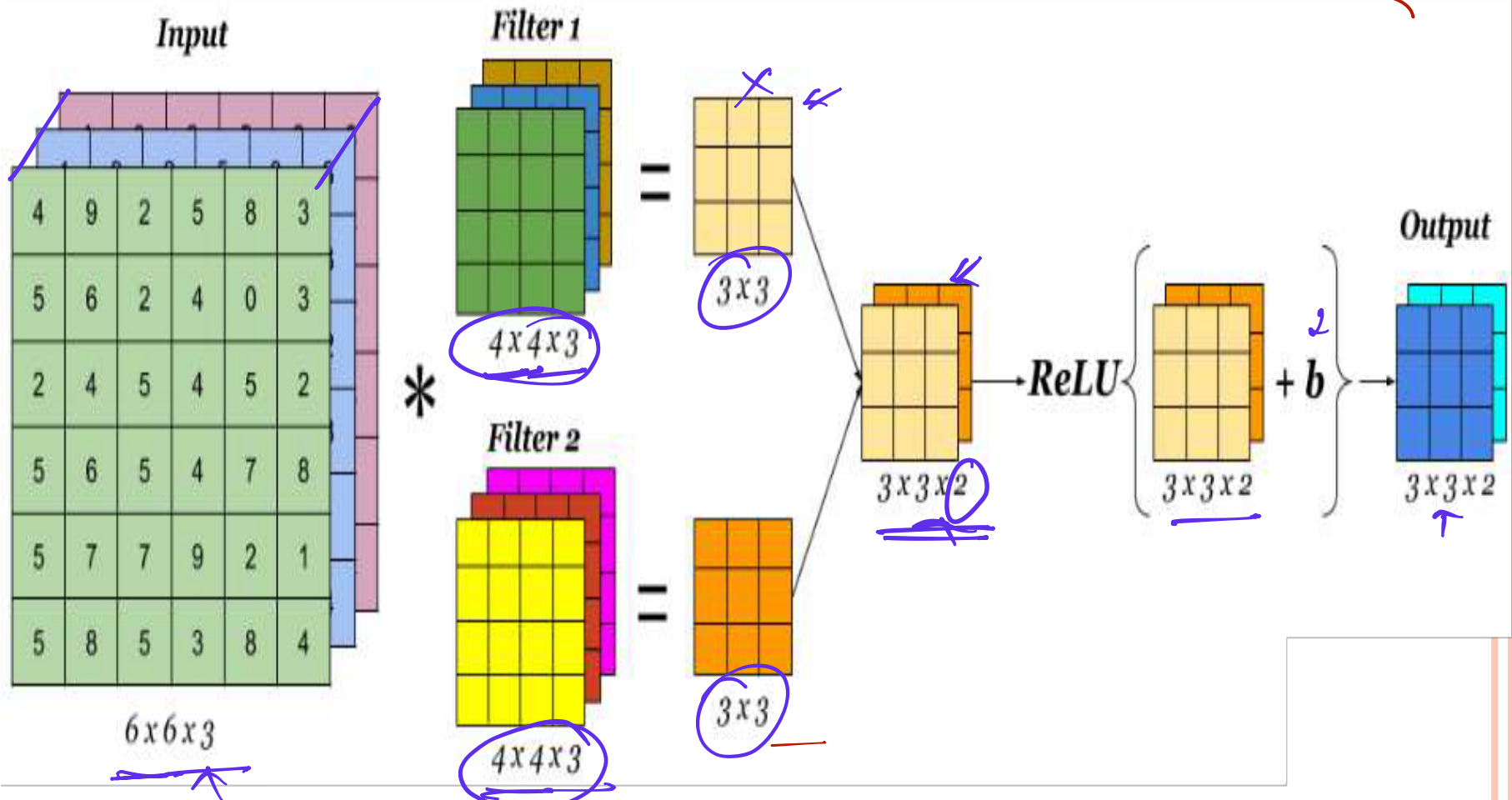
-4	-2
-1	0

Average Pooling

0	1
2	1

Output







THANK YOU!

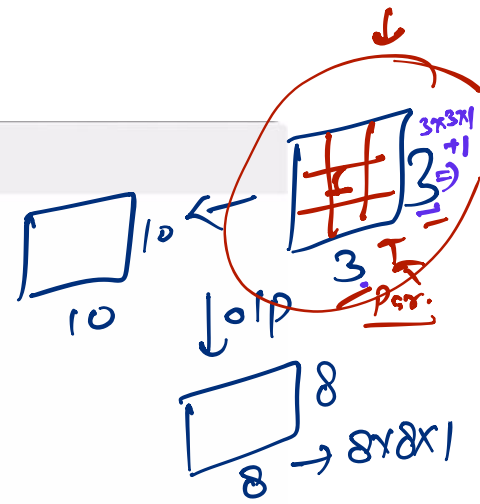
Parameters in a CNN (Example 1)

```
1 my_input = Input(shape = (10, 10, 1))
2 my_output = Convolution2D(1, 3)(my_input)
3 my_output = Activation('sigmoid')(my_output)
4 model = Model(my_input, my_output)
```

```
1 model.summary()
```

parameter
 filter + Bias
 9 values + 1 Bias
 ⇒ 10

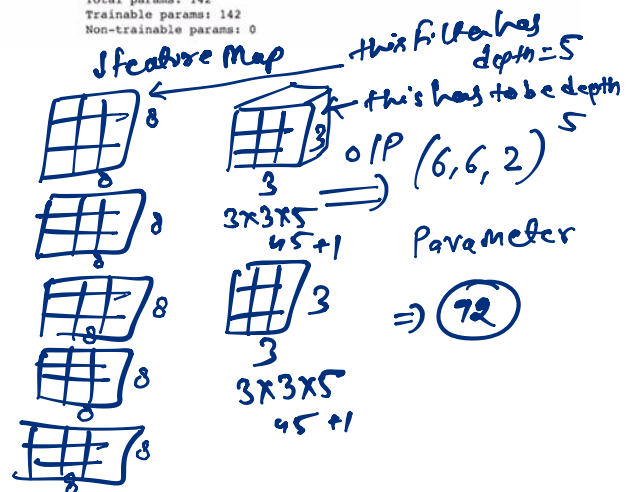
Layer (type)	Output Shape	Param #
input_17 (InputLayer)	[(None, 10, 10, 1)]	0
conv2d_19 (Conv2D)	(None, 8, 8, 1)	10
activation_19 (Activation)	(None, 8, 8, 1)	0
Total params: 10		
Trainable params: 10		
Non-trainable params: 0		



```
1 my_input = Input(shape = (10, 10, 1))
2 my_output = Convolution2D(5, 3)(my_input)
3 my_output = Activation('sigmoid')(my_output)
4 my_output = Convolution2D(2, 3)(my_output)
5 my_output = Activation('sigmoid')(my_output)
6 model = Model(my_input, my_output)
```

```
1 model.summary()
```

Layer (type)	Output Shape	Param #
input_19 (InputLayer)	[(None, 10, 10, 1)]	0
conv2d_22 (Conv2D)	(None, 8, 8, 5)	50
activation_22 (Activation)	(None, 8, 8, 5)	0
conv2d_23 (Conv2D)	(None, 6, 6, 2)	92
activation_23 (Activation)	(None, 6, 6, 2)	0
Total params: 142		
Trainable params: 142		
Non-trainable params: 0		



Parameters in a CNN (Example 3)

```
1 my_input = Input(shape = (100, 100, 3))
2 my_output = Convolution2D(8, 3)(my_input)
3 my_output = Activation('sigmoid')(my_output)
4 my_output = Convolution2D(1, 3)(my_output)
5 my_output = Activation('sigmoid')(my_output)
6 model = Model(my_input, my_output)
```

```
1 model.summary()
```



96, 96, 1

Layer (type)	Output Shape	Param #
input_20 (InputLayer)	[(None, 100, 100, 3)]	0
conv2d_24 (Conv2D)	(None, 98, 98, 8)	224
activation_24 (Activation)	(None, 98, 98, 8)	0
conv2d_25 (Conv2D)	(None, 96, 96, 1)	73
activation_25 (Activation)	(None, 96, 96, 1)	0
Total params: 297		
Trainable params: 297		
Non-trainable params: 0		

$\Rightarrow 3 \times 3 \times 8 = 72 \dots 73$

Parameters in a CNN (Example 4)

```
1 my_input = Input(shape = (100, 5))
2 my_output = Convolution1D(8, 3)(my_input)
3 my_output = Activation('sigmoid')(my_output)
4 my_output = Convolution1D(1, 3)(my_output)
5 my_output = Activation('sigmoid')(my_output)
6 model = Model(my_input, my_output)
```

```
1 model.summary()
```

Layer (type)	Output Shape	Param #
input_25 (InputLayer)	[(None, 100, 5)]	0
conv1d_4 (Conv1D)	(None, 98, 8)	128
activation_30 (Activation)	(None, 98, 8)	0
conv1d_5 (Conv1D)	(None, 96, 1)	25
activation_31 (Activation)	(None, 96, 1)	0
Total params: 153		
Trainable params: 153		
Non-trainable params: 0		